

Příklady na zapouzdření:

1. Implementace vlastní úpravy seznamu.

Cílem je **nepoužívat** funkci *len* (ani její alternativy) pro zjišťování délky seznamu hodnot – vždy využít atribut *počet*. Shoda délky seznamu hodnot a počtu je na zodpovědnosti programátora. V praxi se tento přístup může hodit, protože v určitých implementacích kolekcí dokáže snížit časovou složitost algoritmů metod.

1. Vytvořte třídu *Seznam*.
2. Přidejte soukromé (private) atributy *počet* (int) a *hodnoty* (seznam integerů), které mají implicitní hodnoty (0 a prázdný seznam) – nezadávají se z parametru konstruktoru.
3. Přidejte getter pro atribut *počet*.
4. Přidejte property *je_prazdny*, která vrátí logickou hodnotu „zda neobsahuje hodnoty“.
5. Přidejte metodu *ziskej_prvek*, která vrátí hodnotu uloženou v seznamu na indexu zadaném v parametru. Provádějte kontrolu validity indexu, případně vraťte *None*.
6. Přidejte metodu *prvni_prvek*, která vrátí první prvek. Případně *None*.
7. Přidejte metodu *vypsat*, která vypíše do řádky všechny prvky seznamu *hodnoty*.
8. Přidejte metodu *pridat_prvek*, která na konec seznamu přidá číslo zadané v parametru.
9. Přidejte setter atributu *pocet*, který:
 - a. Pokud je zadaný nový počet menší, zkrátí celý seznam na danou délku.
 - b. Pokud je zadaný nový počet stejný, nedělá nic.
 - c. Pokud je zadaný nový počet větší, naplní chybějící délku číslem 0.
10. Funkčnost třídy vyzkoušejte.

2. Implementace třídy pro řízení odesílání zpráv po síti.

Zprávy se při odeslání musí rozdělit na bloky o zadané délce. Při odesílání se vždy pošle přes síť jedna část zprávy (jeden tzv. packet) a čeká se, než přijde potvrzení od příjemce, že zprávu úspěšně přijal. Pokud se zpráva k příjemci z nějakého důvodu nedostane, odesílatel odeslání packetu opakuje, dokud se nepovede.

Cílem cvičení je vytvoření třídy „pro knihovnu“. Data, která mohou být citlivá na změny nenecháme „z venku“ upravovat. Připravíme metody, aby dané informace o stavu třídy mohl uživatel získat, ale nepovolíme mu změny tam, kde by mohly stav naší třídy narušit.

1. Vytvořte třídu Odesílatel.
2. Přidejte soukromé atributy *délka_bloku* (kladné celé číslo), *zprávy* (seznam řetězců), *packety* (seznam řetězců) a *adresa_příjemce* (řetězec).
3. Přidejte gettery pro atributy *délka_bloku* a *adresa_příjemce*.
4. Přidejte setter pro atribut *adresa_příjemce*. Při změně adresy odstraní z fronty všechny zprávy a packety.
5. Přidejte metodu *přidat_zprávu*, která přebírá novou zprávu jako parametr. Zprávu přidá na konec seznamu *zprávy*.
6. Přidejte **soukromou** metodu *připravit_zprávu*, která ze seznamu zpráv odebere první zprávu a rozdělí ji na bloky dle atributu *délka_bloku*. Tyto bloky vloží do seznamu packetů. Metoda vrací logickou hodnotu sdělující, zda se příprava zprávy zdařila.
7. Přidejte metodu *odeslat_packet*:
 - a. Nejdříve zkontroluje, zda má připravené nějaké packety k odeslání (pokud ne, volá metodu *připravit_zprávu*).
 - b. Pokud již má alespoň jeden packet, „odešle ho“ – knihovnou *time* simulujte dobu trvání.
 - c. S knihovnou *random* simulujte nepovedený příjem zprávy – pokud se „nepovede“, celý pokus o odeslání opakuje, dokud se nepovede příjem.
 - d. Metoda vrací logickou hodnotu sdělující, zda odeslání nakonec uspělo.
8. Přidejte property *zbývající_odeslání*. Z aktuálního stavu třídy spočítá, kolik dohromady zbývá odeslat packetů do vyčerpání všech zpráv ve frontě.
9. Vhodně přidejte do metod výpis činnosti třídy. Např. „Odesílám packet (pokus 4) – text „tohle je packet““ apod.
10. Funkčnost třídy vyzkoušejte.